tea

data prep, brewed cleanly

version 1.6.44

User guide and reference

Mico Mrkaic

August 1, 2026

Abstract. `tea` is a small command-line program for the data-prep-to-quick-estimate loop economists use Stata for: import, clean, generate variables, run regressions, panel methods, MLE, IV, and ARIMA — plus publication-grade SVG graphics and six bundled practice datasets, including the complete IMF World Economic Outlook database. The syntax is close to Stata's where they overlap; the semantics are faithful where it matters (missing-value algebra, the `by:` prefix, gap-aware time-series operators, factor variables). `tea` also runs entirely in a web browser via WebAssembly, with no installation, and its output is byte-identical across machines and numerical libraries. This manual covers installation, a guided tour, every command, the bundled data, and escape hatches to Python/R for things `tea` does not handle.

Audience. Applied economists, policy analysts, and graduate students familiar with Stata. No Stata experience is strictly required, but the manual assumes you know what `regress` is and what panel data looks like.

Status. v1.6.44 follows the v1.0 field-testing release. It has passed 42 regression tests byte-identically on three configurations (native gcc + OpenBLAS in release and sanitizer builds, and WebAssembly clang + reference BLAS), is clean under AddressSanitizer and UndefinedBehavior-Sanitizer, and builds warning-free under `-Wall -Wextra -Werror`. Field testing continues; please report what breaks.

AI-assisted development. This software was developed with the assistance of an AI coding assistant (Anthropic's Claude). The design decisions, mathematical specifications, scope choices, and final acceptance of each component were the author's; the AI contributed substantial portions of the implementation, drafted regression tests, and produced the first draft of this manual. All code was reviewed before inclusion. This manual was likewise drafted with AI assistance and edited by the author. Bugs and design flaws are the author's responsibility.

Contents

Introduction 8
- What tea is . . . 8
- What tea is not . . . 8
- Design choices, locked . . . 8

Installation 9
- Build from source . . . 9
- Dependencies . . . 9
- Sanity check . . . 9
- Running interactively . . . 9

Your first tea session 10

The do-file language 11
- Comments . . . 11
- Macros . . . 11
- Control flow . . . 11
- Capture and quietly . . . 12
- The by: prefix . . . 12

Data management 12
- Importing data . . . 12
- Saving data . . . 13
- Generating and modifying variables . . . 13
- egen extended generators . . . 13
- Type conversion . . . 14
- Labels . . . 14

Exploring data 14
- status . . . 14
- Progress indicator . . . 14
- describe . . . 15
- list . . . 15
- summarize . . . 15
- tabulate . . . 15
- correlate and pwcorr . . . 15
- ttest . . . 16
- codebook . . . 16
- count . . . 16

Reshaping, merging, aggregating 16
- sort and gsort . . . 16
- reshape . . . 16
- preserve / restore . . . 17
- merge . . . 17
- collapse . . . 17
- Frames . . . 17

Time series and panel data 18
- Declaring panel / time structure . . . 18
- xtdescribe . . . 18

Factor variables 18
Atoms . . . 18
Interactions . . . 19
Coefficient naming . . . 19
Known limitation: factor × TS-op composition . . . 19

Linear regression 19
Basic syntax . . . 19
Output . . . 19
Robust and clustered SEs . . . 19
Constraints and hypothesis tests . . . 20

Panel regression 20
Fixed effects . . . 20
Random effects . . . 20
Between effects . . . 20
Hausman test . . . 20
`areg`: absorbed fixed effects . . . 21
`xtivreg, fe`: panel instrumental variables . . . 21
`xtabond`: Arellano-Bond dynamic panel GMM . . . 21

Maximum likelihood estimators 21
`logit` . . . 21
`probit` . . . 21
`poisson` . . . 21
Marginal effects via `margins` . . . 22

Instrumental variables 22
Syntax . . . 22
Example . . . 22
Interpreting the first-stage F . . . 22
Variance estimators . . . 22

Time series models 23
ARIMA . . . 23
Seasonal models: `sarima()` . . . 23
Example . . . 23
`newey`: HAC standard errors . . . 23
Unit-root tests . . . 23
Filters: `tsfilter` . . . 24
VARs, Granger causality, and impulse responses . . . 24
Cointegration rank: `vecrank` . . . 24

State-space models 24
`ucm`: unobserved components . . . 24
`sspace`: general linear state-space models . . . 24
`dfactor`: dynamic factor models . . . 25
`constraint`: linear constraints . . . 25

Post-estimation 25
`predict` . . . 25
Hypothesis tests . . . 26

Output and reproducibility 26

Saving estimates . . . 26
`estout`: publication tables . . . 26
`log` . . . 27

Random numbers and Monte Carlo **27**
Reproducibility . . . 27
Functions . . . 27
A Monte Carlo example . . . 27

Graphics **28**
twoway — overlaid series . . . 28
graph box — grouped box plots . . . 28
Named graphs and graph combine . . . 28

Bundled practice datasets (`sysuse`) **29**
The WEO database . . . 29
The other five . . . 30

The browser edition **30**

Backend-independent output **30**

Escape hatches: when to leave `tea` **31**
When to escape . . . 31
A bridge example . . . 31

Known limitations **31**
Data and metadata . . . 32
Econometric estimators . . . 32
Factor variables . . . 32
Programming . . . 32
Graphics . . . 32

Reporting bugs **32**

Function reference **33**
Arithmetic and rounding . . . 33
Exponential and logarithm . . . 33
Trigonometric and hyperbolic . . . 34
Probability distributions . . . 34
Random number generation . . . 34
Missing-value tests and ranges . . . 34
String construction and inspection . . . 34
String transformation . . . 35
Regular expressions (POSIX extended) . . . 35
Numeric ↔ string conversion . . . 35
Calendar: building dates . . . 35
Calendar: extracting components from a daily date . . . 36
Calendar: converting between frequencies . . . 36
Special: running and time-series-context . . . 36

egen function reference **37**
Group-wise aggregators . . . 37
Position and identity . . . 37

Row-wise aggregators (across columns of a single row) . . . 37

Summarize / tabstat statistics keywords **37**

Stata cheat sheet **38**
Same as Stata . . . 38
Tea-specific differences . . . 38
Stata commands that don't exist in tea . . . 38

Command reference **39**
`twoway` . . . 39
`graph` . . . 39
`scatter` . . . 39
`line` . . . 39
`histogram` . . . 39
`generate` . . . 39
`replace` . . . 39
`egen` . . . 39
`list` . . . 40
`summarize` . . . 40
`tabstat` . . . 40
`count` . . . 40
`describe` . . . 40
`drop` . . . 40
`keep` . . . 40
`rename` . . . 40
`order` . . . 40
`aorder` . . . 40
`label` . . . 40
`format` . . . 41
`sort` . . . 41
`gsort` . . . 41
`tabulate` . . . 41
`tsset` . . . 41
`xtset` . . . 41
`xtdescribe` . . . 41
`xtreg` . . . 41
`hausman` . . . 41
`logit` . . . 41
`probit` . . . 42
`ivregress` . . . 42
`poisson` . . . 42
`arima` . . . 42
`margins` . . . 42
`estimates` . . . 42
`est` . . . 42
`estout` . . . 42
`collapse` . . . 42
`merge` . . . 43
`reshape` . . . 43
`recode` . . . 43
`encode` . . . 43
`decode` . . . 43

destring . . . 43
tostring . . . 43
codebook . . . 43
import . . . 43
export . . . 43
save . . . 44
outreg2 . . . 44
which . . . 44
ssc . . . 44
isid . . . 44
duplicates . . . 44
tempfile . . . 44
tempname . . . 44
constraint . . . 44
dfactor . . . 44
areg . . . 44
xtabond . . . 44
xtivreg . . . 45
sspace . . . 45
ucm . . . 45
var . . . 45
vargranger . . . 45
irf . . . 45
lpirf . . . 45
vecrank . . . 45
newey . . . 45
dfuller . . . 45
pperron . . . 45
tsfilter . . . 45
ttest . . . 45
correlate . . . 46
corr . . . 46
pwcorr . . . 46
file . . . 46
confirm . . . 46
history . . . 46
sysuse . . . 46
use . . . 46
clear . . . 46
error . . . 46
compress . . . 46
status . . . 47
frame . . . 47
regress . . . 47
predict . . . 47
test . . . 47
lincom . . . 47
help . . . 47
pwd . . . 47
cd . . . 47
mkdir . . . 47
dir . . . 47
rmdir . . . 47

erase . . . 47
copy . . . 48
do . . . 48
preserve . . . 48
restore . . . 48
version . . . 48
log . . . 48
exit . . . 48
Native statements . . . 48

Introduction

What `tea` is

`tea` is a C17 program that runs as either an interactive REPL or a batch processor for do-files. It does the data manipulation and econometrics that most applied economists do most days — and nothing else. No plotting, no Bayesian inference, no machine learning. For those, escape to Python or R; see the "Escape hatches" chapter.

The design priority is *Stata fidelity in the overlapping subset.* A do-file that runs in `tea` should run in Stata with the same results, modulo a handful of small documented differences. Where Stata's behavior includes a footgun (`if x > 5` includes missing values, for instance, because missing sorts above every number), `tea` keeps the footgun. That choice is deliberate: it makes `tea` a drop-in for the data-prep phase, even if you eventually move to Stata or R for the final analysis.

What `tea` is not

`tea` v1.0 did not implement graphics; since v1.6.6 it has `scatter`, `line`, `histogram`, multi-series `twoway` (with lowess), `graph box`, and `graph combine`, all on a dependency-free SVG engine. Since v1.6.35 it has a macro time-series inference tier (`newey`, unit-root tests, filters, `var`, `vecrank`, impulse responses), and since v1.6.36 a state-space tier: `ucm` (unobserved components, including damped stochastic cycles), exact-ML `arima` (including multiplicative seasonal models via `sarima()`), `dfactor` (multi-factor dynamic-factor models with optional AR(1) idiosyncratic errors), `sspace` (general linear state-space models with stationary or diffuse initialization), and Stata's `constraint` language. The panel tier includes `areg` (absorbed fixed effects), `xtivreg, fe` (panel IV), and `xtabond` (Arellano-Bond dynamic panel GMM). Still not implemented: GARCH, mixed-effects models, survival analysis, structural breaks, VEC estimation (beyond `vecrank`), Bayesian inference, or machine learning. It also does not implement Stata's full programming language — `tea` has macros, loops, and `capture`, but no `program define`, no `syntax` parser, no Mata.

The `shell` command makes it easy to bridge to Python or R when you need any of the above. See the "Escape hatches" chapter.

Design choices, locked

- **Storage** is one of two types: `double` (numeric, with the full Stata missing-value algebra) or string. Stata's `byte`, `int`, `long`, `float` are cosmetic on read and compressed back on write of `.dta` files.
- **Multiple frames** are supported. Each has its own sort state and panel/time settings.
- **`by:` is strict**. It errors if the data isn't sorted; `bysort` sorts first.
- **`reshape` is in-place**. Use `frame copy` first if you want to keep the original.
- **`m:m` merges are rejected**. They are almost never what the user actually wants.

- **C17**, no novelties. Toolchain stability over newer-standard ergonomics.

Installation

Build from source

You will have received a tarball, `tea-v1.0.tar.gz`.

```
tar xzf tea-v1.0.tar.gz
cd tea
make
```

The build produces `./tea` in the project root. Move it somewhere on your `$PATH` (e.g. `~/bin`) or invoke directly with `./tea`.

Dependencies

You need a C17 compiler (`gcc` 8+ or `clang` 7+) and the following libraries with their development headers:

Library	Why
`libreadline`	line editing, history, tab completion in the REPL
`libopenblas`	matrix kernels
`liblapacke`	LAPACK C interface (regression, ARIMA, IV)
`libgsl`	distributions for *p*-values and confidence intervals
`libreadstat`	Stata `.dta` (and SAS / SPSS) I/O

On Debian / Ubuntu:

```
apt install libreadline-dev libopenblas-dev liblapacke-dev \
            libgsl-dev libreadstat-dev
```

On macOS (Homebrew):

```
brew install readline openblas lapack gsl && make deps-readstat
```

Run `make check-deps` after installing to verify every required header is reachable. If any dependency is missing, that target reports which one and the install command to use.

Sanity check

```
./tea --version           # prints "tea 1.0 -- tiny econometric assistant"
./tea tests/demo.do       # batch: runs a short demo (no errors expected)
make test                 # full regression suite, 39 tests
```

If `make test` reports `tea regression: 39/39 passed`, your build is correct.

Running interactively

Start the REPL by invoking `tea` with no arguments:

```
$ tea
        (  (
         )  )      tea 1.2.2 - tiny econometric assistant
      .........    free Stata-like data analysis at the command line
      |       |]   GPLv3 · Mico Mrkaic · github.com/micomrkaic/tea
```

```
      \       /
       `-----´    type help · sysuse dir for practice datasets · Ctrl-D exits
· _
```

History is persistent at `~/.tea_history` (capped at 2000 lines). Readline editing is available throughout: arrow keys, Ctrl-A/E/K/U/W, Ctrl-L to clear, Up/Down history. Tab completes command names at the start of a line, dataset names after `sysuse`, and variable names elsewhere. `help` lists commands grouped by theme, `help TOPIC` (e.g. `help timeseries`) shows one theme with descriptions, and `help CMD` shows the syntax for one command. Start with `tea -q` to suppress the banner; the browser edition offers the same editing and completion.

Your first tea session

Let's load a small panel dataset, look at it, and run a regression. Save the following as `quickstart.do`:

```
* quickstart.do
import delimited "tests/WEO.csv", clear
rename INDICATOR_ID indicator_id
keep if indicator_id == "NGDP_RPCH"
rename COUNTRY_ID country_id
keep country_id v*
reshape long v, i(country_id) j(year)
rename v growth
keep if !missing(growth)

* Restrict to G7 countries for tractable output
keep if inlist(country_id, "USA", "DEU", "GBR", "FRA", "ITA", "JPN", "CAN")

* Convert string country codes to numeric so we can use factor vars
encode country_id, gen(cid)
xtset cid year

summarize growth
xtdescribe

* Fixed-effects regression of growth on its own lag
xtreg growth L.growth, fe
```

Run it:

```
tea quickstart.do
```

This loads the IMF *World Economic Outlook* CSV (shipped with the distribution), pivots it from wide to long, restricts to G7 countries, sets up the panel structure with `xtset`, and runs a fixed-effects regression. Output looks like Stata's; if you've used Stata before, nothing here should surprise you.

A few things to notice:

- `rename`, `keep if`, `reshape long` all work the same as in Stata.
- `encode` maps string country IDs to consecutive integers 1, 2, 3, . . . and attaches a value label so `list cid` still shows "USA", "DEU", etc.
- `xtset cid year` declares the panel structure. After this, `L.growth` means "growth lagged one year, gap-aware within country."

- `xtreg ..., fe` runs fixed effects (within transformation), with classical SEs by default. Add `vce(robust)` for HC1 or `vce(cluster cid)` for clustered SEs.

The do-file language

A do-file is a sequence of *commands*, one per line, with the following syntax skeleton:

```
[prefix:] command [varlist] [if exp] [in range] [weight] [, options]
```

Comments

Four comment forms are supported:

```
* Whole-line comment starting with asterisk
// Inline comment to end of line
/* Block comment
   spanning multiple lines */
generate x = 1 ///
       + 2     // line continuation: combine the next line with this one
```

Macros

Local macros are referenced as “`` `name' ``” (back-tick … tick); global macros as `$name`.

```
local x = 5
display "x = `x'"              // x = 5

global path "/home/me/data"
use "$path/employment.dta", clear

local vars "gdp pop unemp"
foreach v of local vars {
    summarize `v'
}
```

After most estimation commands, results are stored in the `e()` namespace:

```
regress y x1 x2
display e(N)         // sample size
display e(r2)        // R-squared
display _b[x1]       // x1's coefficient
display _se[x1]      // x1's standard error
```

Summarize stores results in `r()`:

```
summarize gdp
display r(mean)
display r(sd)
```

Control flow

```
* Conditional
if `n' > 100 {
    display "large sample"
}
else {
```

```
    display "small sample"
}

* Foreach over a list, varlist, or numlist
foreach v of varlist gdp pop unemp {
    quietly summarize `v'
    display "`v': N = " r(N) ", mean = " r(mean)
}

* Forvalues
forvalues i = 1/5 {
    display "iteration `i'"
}
```

Capture and quietly

`quietly` suppresses output; `capture` additionally suppresses errors (storing the return code in `_rc`):

```
quietly regress y x       // runs, no output
display _b[x]              // coefficient is still available

capture confirm variable z // doesn't error if z doesn't exist
if _rc != 0 {
    display "z is missing"
}
```

The `by:` prefix

```
sort country year                // required before by:
by country: generate dy = y - y[_n-1]

bysort country (year): generate y_first = y[1]   // sorts first
```

`tea` enforces the sort requirement — using `by:` on unsorted data is a hard error, not a silent miscomputation. Use `bysort` when you want it to sort automatically.

Data management

Importing data

`tea` reads CSV, TSV, Stata `.dta`, Excel `.xlsx`, and OpenDocument `.ods` files, plus its own native `.tea` binary format. As of v1.6.7 the native format (TEA2) carries value-label sets with the dataset, exactly like `.dta`; legacy TEA1 files still read, minus value-label attachments. `use`, `save`, and `merge ... using` all default a missing extension to `.dta`.

```
import delimited "data.csv", clear      // CSV
import delimited "data.tsv", clear      // TSV (auto-detected)
import excel "data.xlsx", firstrow clear
import excel "data.xlsx", sheet("Sheet2") firstrow clear
import excel "WPP2024.xlsx", sheet("Estimates") ///
    cellrange(A17:AF22000) firstrow case(lower) clear
```

Excel import is NATIVE: tea reads .xlsx itself (including cached formula values and multi-sheet workbooks) with no external converter — identically on Linux, macOS, and in the browser, where a dragged-in workbook imports directly. Only .ods still uses ssconvert/libreoffice.

`cellrange(A17[:AF22000])` restricts an Excel import to a sheet rectangle — real workbooks (the UN WPP file) carry title junk above the table, and with `firstrow` the range's FIRST row is the header row. The end corner is optional. `case(preserve|lower|upper)` folds the resulting names (default preserve for Excel, lower for delimited, as in Stata); the quoted form `case("lower")` is accepted. For text files the same rectangle restriction is `rowrange(r1[:r2])` and `colrange(c1[:c2])` on `import delimited`. Slicing is CSV-aware: quoted cells containing commas, doubled quotes, or embedded newlines count as single fields.

```
use "data.dta", clear                    // Stata format
use "data.tea", clear                    // native tea format
```

`tea` auto-detects the delimiter for `.csv` and `.tsv`; if the file extension is ambiguous, pass `delimiter(",")` or `delimiter(tab)` explicitly.

The `firstrow` option (for Excel) tells `tea` to use the first row as variable names; invalid characters are removed (`Country Code` → `CountryCode`) and a header that is empty, a duplicate, or starts with a digit (`1960`) falls back to the Excel column letter (`F`, `BQ`, . . .) — exactly Stata's rule. Without `firstrow`, columns are named `A`, `B`, . . . and row 1 is kept as a data row, again as in Stata; columns mixing a text header with numeric data become string columns.

`import delimited` follows Stata's naming rule for CSV/TSV headers: names are lowercased and invalid characters removed (`Country Code` → `countrycode`); a header that is empty, a duplicate, or starts with a digit becomes the position name `v#`. Use `case(preserve)` (or `case(upper)`) to override the case folding.

CSV import handles RFC 4180–style quoted fields, including embedded commas and newlines.

Saving data

```
save "out.dta", replace            // Stata format (default for .dta)
save "out.tea", replace            // native tea format
export delimited "out.csv", replace
export delimited "out.tsv", replace
```

Format choice. Use `.dta` if the data will be read by Stata. Use `.tea` if it's a temporary file between `tea` sessions (faster to read/write, no readstat overhead). Use `.csv` or `.tsv` if a non-statistical tool will consume the data.

Generating and modifying variables

```
generate gdp_log = log(gdp)
generate growth = (gdp - gdp[_n-1]) / gdp[_n-1]
replace growth = . if year == 1990             // missing for first year

generate str8 region = "WEST"
replace region = "EAST" if country_id == "CHN"
```

The `str#` prefix declares a string variable (the number is the maximum width, used by Stata when writing `.dta` files; in `tea` it's cosmetic on read but compressed back on write).

`egen` extended generators

`egen` provides aggregate functions that `generate` can't do in a single expression:

```
egen mean_gdp     = mean(gdp), by(country)
egen total_pop    = total(pop)
egen rank_gdp     = rank(gdp)
egen group_id     = group(country region)         // assigns 1..K
```

```
egen row_total     = rowtotal(x1 x2 x3 x4)
egen has_any       = anyvalue(x), values(1 5 9)
```

The full list of functions: `mean`, `sum`, `total`, `count`, `min`, `max`, `sd`, `var`, `median`, `group`, `rank`, `rowtotal`, `rowmean`, `rowmin`, `rowmax`, `rowmiss`, `rownonmiss`, `tag`, `seq`, `anyvalue`, `concat`.

Type conversion

```
* Numeric -> string
tostring year, generate(year_str)

* String -> numeric (errors if non-numeric values present)
destring score, generate(score_n)
destring score, generate(score_n) force      // converts non-numeric to .

* String -> integer codes + value label
encode country_id, generate(cid)             // alphabetical 1..K

* Integer codes -> string (inverse of encode)
decode cid, generate(country_str)

* Recoding numeric values
recode age (0/17 = 1 "minor") (18/64 = 2 "working") (65/max = 3 "senior"), gen(age_cat)
```

Labels

```
label data "World Economic Outlook subset"
label variable growth "Real GDP growth (%)"

label define region 1 "North" 2 "South" 3 "East" 4 "West"
label values region region
```

Exploring data

status

A one-line summary of what's in memory — a tea extension (Stata has no single equivalent):

```
. status
WB_final.dta - 25,808,384 obs, 7 vars, 1.9 GB  (sorted: CountryCode year; xtset: ifscode year)
```

The source name is tracked through `use` / `import` / `sysuse`, updated by `save` (as Stata's "Contains data from" is), and cleared by `clear`. The memory figure is the exact in-memory data size: 8 bytes per numeric cell, pointer plus string length per string cell.

Progress indicator

Long operations (import, reshape, merge) show a live progress line — another tea extension, useful because tea runs on one CPU and big jobs can otherwise look hung:

```
reshape long: 42% (10,800,000/25,808,384)
```

External conversion work of unknown length — the ssconvert/libreoffice phase of `import excel`, which dominates on big workbooks — shows an activity form instead, a spinner with elapsed time:

```
converting WB_data.xlsx / 34s
```

Do-file execution is quiet by default; `set echo on` prints each line Stata-style (`. <line>`) before its output — real Stata echoes `do` by default, a documented tea deviation in favor of stable batch output. The browser editor's Run button uses it, so the web terminal shows commands with their results.

Long interactive output can be paged, exactly Stata's `more`: after `set more on`, `list` and `describe` pause at each screenful with `--more--` (any key = next page, Enter = one line, q = stop). Only the interactive REPL with a terminal ever pages; do-files, pipes, and logs always stream. Default is off, as in modern Stata.

It is deliberately invisible everywhere it could interfere: nothing is drawn until an operation has run for about a second (short operations produce zero output), it goes to stderr only when stderr is a terminal (logs, pipes, batch runs, and `capture` never see it), and the line is erased on completion so nothing lands in scrollback. `set progress off` disables it entirely.

describe

```
describe                          // all variables
describe gdp pop                  // just these two
describe gdp*                     // wildcards work
```

Shows variable name, storage type, display format, and variable label.

list

```
list                              // all rows (capped at 200)
list in 1/10                      // first 10 rows
list country year gdp if year == 2020
```

Column widths auto-fit to the maximum of header and value widths.

summarize

```
summarize                         // all numeric variables
summarize gdp, detail             // adds percentiles, skewness, kurtosis
summarize gdp if year >= 2000
bysort country: summarize gdp
```

After `summarize`, the following `r()` macros are set: `r(N)`, `r(mean)`, `r(sd)`, `r(min)`, `r(max)`, `r(sum)`, `r(Var)`. With `detail`, also `r(p1)`, `r(p5)`, `r(p10)`, `r(p25)`, `r(p50)`, `r(p75)`, `r(p90)`, `r(p95)`, `r(p99)`, `r(skewness)`, `r(kurtosis)`.

tabulate

```
tabulate region                              // one-way frequency table
tabulate region year, row column cell        // two-way + percentages
tabulate region, summarize(gdp) means        // means of gdp by region
```

correlate and pwcorr

```
correlate v1 v2 v3 [, means covariance]
pwcorr v1 v2 v3 [, sig obs]
```

`correlate` uses listwise deletion (one common sample for the whole matrix, Stata's behavior); `pwcorr` computes each pair on its own maximal sample. `means` prefixes the summary block; `covariance` prints the covariance matrix instead.

ttest

```
ttest v == 3            // one-sample
ttest v1 == v2          // paired
ttest v, by(group)      // two-sample (pooled; add unequal for Welch)
```

Full Stata table layout, including the three-alternative footer.

codebook

```
codebook                          // all variables
codebook gdp                      // just one
```

Reports type, unique values, missing count, range, and distribution summary. More compact than `summarize, detail` for screening a new dataset.

count

```
count                             // total observations
count if missing(gdp)
count if gdp > 0 & year >= 2000
```

After `count`, `r(N)` holds the count.

Reshaping, merging, aggregating

sort and gsort

```
sort country year                 // ascending on both
gsort -gdp                        // descending on gdp
gsort country -year               // mixed
```

reshape

`reshape` is in-place — to keep the original you must `frame copy` (or `preserve`) first.

```
* Wide to long: each yY variable becomes y at year Y
reshape long y, i(country) j(year)

* Long to wide
reshape wide y, i(country) j(year)

* Multiple stubs
reshape long y z, i(country) j(year)

* String j: levels are name suffixes (long) or column-name parts (wide)
reshape wide y, i(country year) j(indicator_code) string
reshape long y, i(country year) j(indicator_code) string
```

Variables that are not part of a stub and not in `i()` are **carried along**, exactly as in Stata: going long they are replicated across the j-rows of each wide observation; going wide they must be constant within each `i()` group (error r(9) otherwise — drop them or add them to `i()`). Formats and variable labels survive on `i()` and carried columns.

Going wide, every generated column name (`stub` + j value) must be a valid identifier; if your j values contain characters like `.`, clean them first with `strtoname()`:

```
gen code_safe = strtoname(indicator_code)
reshape wide y, i(country year) j(code_safe) string
```

`reshape wide` also insists that j is never missing/empty and never repeats within an `i()` group — both are data errors in Stata, and tea fails loudly rather than guessing.

`preserve` / `restore`

`preserve` snapshots the data in memory; `restore` brings the snapshot back. One level deep, as in Stata. The snapshot lives on disk as a native `.tea` tempfile, so preserving a large dataset does not double peak memory.

```
preserve
keep indicator_code indicator_name
duplicates drop
save "lookup.dta", replace
restore
```

Variants: `restore, not` discards the snapshot without reloading; `restore, preserve` reloads but keeps the snapshot for another `restore`. A preserve still pending when a do-file concludes — normally or via an abort — is restored automatically, with a note (Stata behavior).

`merge`

```
merge 1:1 country year using "macro.dta"
merge m:1 country      using "country_attrs.dta"
merge 1:m country year using "panel_history.dta"
```

After `merge`, the variable `_merge` encodes the match status: 1 = master only, 2 = using only, 3 = matched.

`m:m` is deliberately rejected. If you think you need `m:m`, you almost certainly want a different operation (`joinby`-style cross product, or an explicit deduplication before merging).

`collapse`

```
collapse (mean) gdp pop (sum) revenue (sd) sd_gdp = gdp, by(country)
collapse (median) gdp [aweight=pop], by(region year)
```

Available aggregators: `mean`, `median`, `sum`, `min`, `max`, `sd`, `count`, `first`, `last`, `p25`, `p50`, `p75`, `p10`, `p90`.

Frames

A frame is a named in-memory dataset. Multiple frames let you keep several datasets loaded at once.

```
frame create macro
frame change macro
use "macro.dta", clear

frame change default
* now we're back on the original frame

frame copy default backup    // duplicate default into a new frame
frame drop backup
```

Time series and panel data

Declaring panel / time structure

```
tsset year                        // pure time series
xtset country year                 // panel
xtset country year, yearly         // explicit time unit
```

After this, time-series operators are valid:

Operator	Meaning
`L.x`	first lag (one period back)
`L2.x`	second lag
`L(1/3).x`	first through third lags
`F.x`	first lead
`D.x`	first difference
`D2.x`	second difference (Stata's notation)
`S.x`	seasonal difference

These can mix with factor variables (with one limitation; see the "Factor variables" chapter):

```
regress y L.y x i.region              // OLS with lag and region dummies
xtreg y L(1/3).y x, fe                 // panel FE with three lags
```

`tea`'s time-series operators are *gap-aware*: if a country has data for 1995 but not 1996, `L.y` for 1997 returns missing (not the 1995 value).

`xtdescribe`

```
xtdescribe
```

Reports number of panels, observations per panel (min/avg/max), and sample pattern. Useful for checking whether a panel is balanced.

Factor variables

Stata's factor-variable grammar lets you include categorical regressors and their interactions inline, without manually generating dummies. `tea` supports the same grammar across every estimator (`regress`, `xtreg`, `logit`, `probit`, `poisson`, `ivregress`).

Atoms

Token	Meaning
`i.var`	dummies for each non-base level; base = smallest level
`ib2.var`	same, with base level explicitly set to 2
`c.var`	continuous regressor (mostly for use inside `#`)

```
regress wage exper i.region             // region dummies
regress wage exper ib2.region           // base = region 2
```

`i.var` requires `var` to be numeric. For string identifiers (e.g. ISO country codes), use `encode` first:

```
encode country_id, gen(cid)
regress y x i.cid
```

Interactions

Token	Meaning
A#B	cross product: $(K_A - 1) \times (K_B - 1)$ columns
A##B	main effects + interaction (equivalent to A B A#B)

```
regress wage i.region#c.exper           // region-specific slopes
regress wage i.region##c.exper          // adds main effects of region and exper
regress wage i.region#i.year            // region-by-year interaction
```

The `c.` prefix tells `tea` to treat the variable as continuous in an interaction — without it, a numeric variable would be expanded as a factor.

Coefficient naming

Factor expansions produce coefficients with structured names that `predict`, `margins`, and `_b[]` all understand:

```
regress wage i.region
display _b[2.region]                     // coefficient on region 2 dummy
display _b[3.region#c.exper]             // interaction coefficient
```

Known limitation: factor × TS-op composition

Tokens like `i.country#c.L.gdp` (factor interaction with a lagged regressor) are **not** supported in v1.0.

Workaround: pre-compute the lag manually and use the bare variable inside the interaction:

```
generate L1_gdp = L.gdp
regress y i.country#c.L1_gdp               // works
```

Linear regression

Basic syntax

```
regress y x1 x2 x3
regress y x1 x2, vce(robust)              // HC1 robust SEs
regress y x1 x2, vce(cluster country)     // CR1 clustered SEs
regress y x1 x2 [fweight=n], vce(robust)  // frequency weights
```

Weight types: `fweight` (frequency, integer), `aweight` (analytic, rescaled to sum to N), `pweight` (probability, no rescaling), `iweight` (importance, like `aweight` but no rescaling).

Output

A standard ANOVA table with F-statistic and R^2, followed by the coefficient table with t-statistics, p-values, and 95% confidence intervals. After `regress`, the following are stored:

- `e(N)`, `e(r2)`, `e(r2_a)`, `e(rmse)`, `e(F)`, `e(p)`, `e(df_r)`, `e(df_m)`
- `_b[varname]` and `_se[varname]` for each coefficient, plus `_b[_cons]` and `_se[_cons]`.

Robust and clustered SEs

The implementation uses HC1 for robust and CR1 for clustered, matching Stata's defaults. Cluster variables may be string (panel IDs) or numeric.

Constraints and hypothesis tests

After `regress`, use `test` for joint hypotheses and `lincom` for linear combinations:

```
regress y x1 x2 x3 x4

test x1 = x2                                    // single restriction
test x1 = 0, x2 = 0                             // joint (F-test)
test x1 x2 x3                                   // all = 0 jointly

lincom x1 + x2                                   // linear combination + CI
lincom 0.5*x1 + 0.5*x2 - x3
```

Panel regression

Fixed effects

```
xtset country year
xtreg y x1 x2, fe                                // within (default for xtreg)
xtreg y x1 x2, fe vce(cluster country)
```

Output includes:

- Within / between / overall R^2
- σ_u, σ_e, ρ (variance components)
- F-test of $u_i = 0$
- `corr(u_i, Xb)` — informally measures the FE–RE departure

Identifies as `e(cmd) = "xtreg"` with `e(model) = "fe"`.

Random effects

```
xtreg y x1 x2, re                                // Swamy-Arora FGLS
```

Implementation: Swamy-Arora variant (the Stata default). Two-step FGLS — first estimate σ_u^2, σ_e^2 from the within and between residuals; then quasi-demean using $\theta = 1 - \sigma_e/\sqrt{T\sigma_u^2 + \sigma_e^2}$.

Between effects

```
xtreg y x1 x2, be                                // OLS on panel means
```

Collapses the data to one observation per panel (the panel means of y and each x), then runs OLS. This is the BE estimator that underpins one of the components of the RE-FE decomposition.

Hausman test

```
quietly xtreg y x1 x2, fe                        // populates fe_est
quietly xtreg y x1 x2, re                        // populates re_est
hausman                                           // tests FE vs RE
```

Order of `xtreg fe` and `xtreg re` doesn't matter; `tea` keeps both in dedicated workspace slots and the `hausman` command (no arguments needed) tests their coefficient difference.

areg: absorbed fixed effects

```
areg y x1 x2, absorb(groupvar) [robust cluster(v)]
```

The within transform on `absorb()` groups: coefficients are identical to including a full set of group indicators (verified in the test suite to every printed digit, standard errors included), but the indicators are never materialized. Degrees of freedom are the dummy-equivalent `N - K - G`, so classical, robust, and clustered standard errors all match the dummy regression's.

xtivreg, fe: panel instrumental variables

```
xtivreg y [exog] (endog = instruments), fe [robust]
```

Within transform (panel demeaning) of every variable, then 2SLS on the demeaned system, with `df = N - K - N_g` for the absorbed means. `robust` clusters on the panel variable (Stata's convention for `xtivreg, fe`). Only the fe estimator is implemented; `re` is staged.

xtabond: Arellano-Bond dynamic panel GMM

```
xtabond y [x...], lags(1) [maxldep(#) twostep robust]
```

Difference GMM for dynamic panels: the model in first differences, with GMM-style (non-collapsed) instruments — at time t, the levels $y_{i1}, \ldots, y_{i,t-2}$, optionally capped by `maxldep()` — and strictly exogenous regressors instrumented by their own differences. One-step estimation uses the Arellano-Bond H weight (tridiagonal $2, -1$); `twostep` re-weights with the one-step residual moments. `robust` gives the panel-clustered sandwich. The Sargan statistic is reported and posted as `e(sargan)`.

Honest caveats, printed where relevant: the Windmeijer correction for two-step standard errors is staged (two-step conventional SEs are known to be downward biased); the AR(1)/AR(2) autocorrelation tests are staged; only `lags(1)` is accepted in this release. The implementation is verified against an independently written GMM referee to every reported digit (see `KNOWN_BUGS.md`, v1.6.42).

Maximum likelihood estimators

`tea` implements logit, probit, and Poisson regression via a shared Newton-Raphson driver. All three support `if`, `in`, weights, and `vce(robust|cluster)`.

logit

```
logit voted income education
logit y x1 x2 i.region, vce(cluster country)
```

Reports the maximum likelihood iteration trace, LR χ^2 test against the constant-only model, pseudo R^2, and the coefficient table with z-statistics.

probit

Same syntax, normal CDF instead of logistic CDF:

```
probit y x1 x2 i.region
```

poisson

For count outcomes:

```
poisson accidents speed_limit population, vce(cluster state)
```

The link is $\log E[y \mid x] = x'\beta$; coefficients are log-rate-ratios. $y < 0$ is rejected. Non-integer y is permitted (treated as the GLM extension).

Important: `tea`'s Poisson drops the $\log(y!)$ constant in the log-likelihood, so the absolute `e(ll)` value differs from Stata's, but the coefficient estimates, SEs, and likelihood-ratio tests are identical.

Marginal effects via `margins`

After any of the above, `margins` produces average or at-the-mean marginal effects:

```
logit y x1 x2 i.region
margins, dydx(*)                            // average marginal effects
margins, dydx(x1)                           // just x1
margins, dydx(*) atmeans                    // at-the-means MEs
```

Standard errors use the delta method.

Instrumental variables

`ivregress 2sls` implements two-stage least squares with a first-stage F diagnostic for weak instruments.

Syntax

```
ivregress 2sls y [exog_vars] (endog_vars = instruments) [if] [in] [weight] [, options]
```

The parenthesized group is the key piece: variables to the left of the `=` are endogenous; variables to the right are excluded instruments.

Example

```
* Wage equation: education is endogenous, parents' education is the IV
ivregress 2sls wage exper exper2 (educ = momeduc dadeduc), vce(robust)
```

Output includes the coefficient table (with z-statistics from asymptotic normality), the list of instrumented variables, the list of instruments (included exogenous + excluded), and the first-stage F-statistic for the weakest endogenous regressor.

Interpreting the first-stage F

A common rule of thumb (Stock-Yogo): if the first-stage F is below 10, the instrument is weak and the IV estimates may have substantial finite-sample bias. `tea` flags this in the output with a caret.

Variance estimators

```
ivregress 2sls y (x = z)                            // classical
ivregress 2sls y (x = z), vce(robust)               // HC1
ivregress 2sls y (x = z), vce(cluster country)      // CR1
```

The sandwich form is $V = A \cdot B \cdot A$ where $A = (\hat{X}'\hat{X})^{-1}$ (using fitted endogenous columns) and B uses residuals computed from the *original* X (this is the correct 2SLS variance — not the naive OLS-on-fitted variance).

Time series models

ARIMA

```
tsset year
arima y, arima(p d q) [noconstant]
arima y x1 x2, arima(1 0 1)                    // ARMAX with two exog
```

Specifies an ARIMA(p, d, q) model: AR order p, difference order d, MA order q. Exogenous regressors can be included (then the model is ARMAX).

Method (since v1.6.37). `arima` computes *exact maximum likelihood* via the state-space engine: the ARMA process on the differenced, mean-adjusted series is cast in companion form and the exact Gaussian likelihood is evaluated by the Kalman filter with stationary (Lyapunov) initialization — the same approach as Stata. Stationarity and invertibility are enforced during optimization through the partial-autocorrelation transform. Conventions that still differ from Stata (see COMPATIBILITY.md):

- Standard errors are OIM (observed information); Stata defaults to OPG/BHHH. Point estimates and log likelihoods are the comparable quantities and match exact-ML references to reported precision.
- Standard errors for constants/exog in ARMAX are asymptotic.

Seasonal models: `sarima()`

```
arima lp, arima(0 1 1) sarima(0 1 1 12)        // the airline model
```

Multiplicative seasonal ARIMA$(p, d, q) \times (P, D, Q)_s$: the seasonal polynomials multiply the regular ones and the product is expanded onto the same exact-ML engine; seasonal differencing is applied after regular differencing. Limits: $p + Ps \leq 15$ and $q + Qs \leq 15$. The canonical airline fit matches the exact-ML references to reported precision (test 75). Note that tea, like Stata, includes a constant unless `noconstant`; statsmodels' SARIMAX defaults to no trend, so likelihood comparisons should use `noconstant`.

Example

```
* US growth ARIMA(1,0,0)
tsset year
arima growth, arima(1 0 0)
```

The output groups coefficients into "ARMA model" (constant + exog), "AR" (ϕ_i), "MA" (θ_j), and `/sigma` (the innovation standard deviation).

`newey`: HAC standard errors

```
newey y x1 x2, lag(#)
```

OLS point estimates with Newey-West (Bartlett kernel) standard errors; `lag(0)` reproduces White/robust.

Unit-root tests

```
dfuller y [, lags(#) trend]
pperron y [, lags(#) trend]
```

Augmented Dickey-Fuller and Phillips-Perron. MacKinnon-style critical values are tabulated; the reported approximate p-value uses an interpolation that can differ from Stata's in the third decimal (documented in COMPATIBILITY.md).

Filters: tsfilter

```
tsfilter hp new = y [, smooth(#)]
tsfilter bk new = y [, minperiod(#) maxperiod(#)]
tsfilter hamilton new = y [, h(#) p(#)]
```

Hodrick-Prescott, Baxter-King, and Hamilton-regression filters; each creates the cyclical component under the new name.

VARs, Granger causality, and impulse responses

```
var y1 y2 [y3 ...], lags(#)
vargranger
irf create, step(#)
irf table [irf|oirf]
lpirf y1 y2, step(#) [lags(#)]
```

`var` estimates a reduced-form VAR by per-equation OLS; `vargranger` reports the Granger exclusion Wald tests; `irf` computes (orthogonalized) impulse responses from the last VAR; and `lpirf` estimates local-projection impulse responses (Jorda) with Newey-West bands. One documented convention difference: tea's posted `Sigma` uses the ML divisor T (Stata reports the same matrix; see COMPATIBILITY.md for the small-sample discussion).

Cointegration rank: vecrank

```
vecrank y1 y2 [y3 ...], lags(#)
```

Johansen trace statistics with the standard critical-value table. Full VECM estimation is not implemented; `vecrank` answers the rank question and the "Escape hatches" chapter covers the rest.

State-space models

Everything in this chapter runs on one engine (`src/kalman.c`): a univariate-sequential Kalman filter with exact diffuse initialization (Durbin-Koopman), a matching smoother, and exact maximum likelihood throughout. `arima` uses the same engine. Standard errors are OIM (observed information via the numerical Hessian of the exact likelihood, delta method for transformed parameters); Stata defaults to OPG for some of these commands, so point estimates and log likelihoods are the comparable quantities. Every estimator here is verified against independent references in the regression suite (tests 72-75) — statsmodels where its optimizer succeeds, and the large-kappa limit of the diffuse likelihood where it does not (see COMPATIBILITY.md for two documented cases where tea's answer is the verifiably correct one).

ucm: unobserved components

```
ucm y, model(ntrend|llevel|lltrend|rwalk|rwdrift)
       [seasonal(#) cycle smstate(NEW)]
```

Harvey structural models: the trend family, an optional dummy seasonal of period `#`, and since v1.6.41 an optional damped stochastic cycle (`cycle`), reported as `damping`, `frequency`, and `var(cycle)` with delta-method SEs. `smstate(NEW)` saves the smoothed level. The Nile local level reproduces the Durbin-Koopman book values exactly.

sspace: general linear state-space models

```
constraint 1 [y]u = 1
sspace (u L.u, state) (y u, noerror noconstant),
```

```
constraints(1) covstate(diagonal) [diffuse smstates(stub)]
```

State equations (marked `, state`) are linear in lag-1 states; observation equations are linear in contemporaneous states, with optional constants and per-equation `noerror`. Identification is the user's job through the `constraint` subsystem, exactly as in Stata. `covstate(identity)` (the default) fixes state disturbance variances at 1; `covstate(diagonal)` frees them. Stationary models initialize by Lyapunov solve with a positive-definiteness guard; `diffuse` switches to exact diffuse initialization and lifts the stationarity requirement (a tea extension — Stata's sspace has no diffuse option). Higher lags are written with auxiliary identity states. The AR(1) written as `sspace` reproduces `arima` to every reported digit, standard error included.

`dfactor`: dynamic factor models

```
dfactor (y1 y2 y3 = [exog] [, noconstant ar(1)]) (f1 [f2] = , ar(p))
```

Up to four factors with full interacting VAR(p) dynamics (p at most 4), `Var(v) = I` normalization, optional AR(1) idiosyncratic errors in the observation equations (`ar(1)` in the observation-equation options), and `smfactor(stub)` for smoothed factors. Sign convention: the first loading on each factor is kept positive. Multi-factor models generally need identifying constraints (`constraints()`), the reason the constraint subsystem exists.

`constraint`: linear constraints

```
constraint 1 [y]u = 1
constraint 2 [f1]L.f2 = [f2]L.f1
constraint list
constraint drop 1
```

Stata's constraint language: definitions are stored per session, referenced by `constraints(numlist)` in estimation commands, applied by reparameterizing onto the null space (so the optimizer never sees the constrained directions), and constrained coefficients print `(constrained)` in tables. Variance parameters cannot be constrained in this release.

Post-estimation

`predict`

After any estimator, `predict` generates a new variable with the model prediction or a residual:

```
* After regress
regress y x1 x2
predict yhat                        // linear prediction (default: xb)
predict resid, residuals            // residuals
predict resid, r                     // same, shorter

* After logit/probit
logit voted income
predict p                            // Pr(y=1)
predict xb, xb                       // linear index

* After xtreg fe
xtreg y x1, fe
predict u, u                         // estimated u_i
predict e, e                         // idiosyncratic residual
predict ue, ue                       // u_i + e_it
```

Hypothesis tests

`test` performs Wald tests on linear restrictions:

```
regress y x1 x2 x3
test x1 = 0                                    // single restriction
test x1 = 1, x2 = 0                            // joint restriction
test x1 x2 x3                                  // x1 = x2 = x3 = 0
```

`lincom` computes linear combinations with their SEs and 95% CIs:

```
lincom x1 + x2 - x3
lincom 0.5*x1 + 0.5*x2
```

Output and reproducibility

Saving estimates

The `estimates` (alias `est`) command suite saves and manages named estimation results:

```
regress y x1
estimates store m1

regress y x1 x2
estimates store m2

regress y x1 x2 x3
estimates store m3

estimates dir                         // list stored estimates
estimates table m1 m2 m3, stats(N r2 rmse) star
estimates restore m1                  // load m1 back to last_est
estimates drop m2                     // remove m2
estimates drop _all                   // remove all
```

`estout`: publication tables

For papers, use `estout` to produce LaTeX, markdown, or plain side-by-side tables:

```
estout m1 m2 m3                                       // default: LaTeX
estout m1 m2 m3, format(latex) stats(N r2 rmse)
estout m1 m2 m3, format(markdown) se star
estout m1 m2 m3, format(plain) t

estout m1 m2 m3 using "table.tex", stats(N r2)       // write to file
estout m1 m2 m3, title("Regression results") label(tab:main)

* Subset of coefficients
estout m1 m2, keep(x1 x2 _cons) stats(N r2)
estout m1 m2, drop(_cons)                            // drop constant row
```

Options:

Option	Effect
`format(latex|markdown|plain)`	output format (default: `latex`)

Option	Effect
`se`, `t`, `p`	show SEs / *t*-stats / *p*-values below coefs
`star`	add significance stars (10%, 5%, 1%)
`stats(N r2 r2_a rmse F ...)`	footer rows
`title(...)`, `label(...)`	LaTeX caption + reference label
`keep(...)`, `drop(...)`	restrict coefficient rows
`nomtitles`, `mtitles(...)`	override per-model column labels
`using FILE`	write to file instead of stdout

LaTeX output produces a stand-alone `tabular` block you can `\input` in your paper.

log

```
log using "session.log", replace
* commands and their output are captured...
log close
```

Captures both the commands you ran (echoed with a `.` prefix) and the output they produced.

Random numbers and Monte Carlo

Reproducibility

```
set seed 12345                              // fix the PRNG state
```

`tea` uses a 64-bit PCG generator. `set seed N` is deterministic — the same seed always produces the same sequence, across runs, machines, and `tea` versions.

Functions

```
generate u = runiform()                     // Uniform[0, 1)
generate z = rnormal()                      // Standard normal
generate w = rnormal(5, 2)                  // Normal(mean=5, sd=2)
```

Normal draws use Box-Muller with caching. All three functions are available in any expression context (`generate`, `replace`, `egen rowfirst/rowlast`, etc.).

A Monte Carlo example

```
* OLS slope sampling distribution under known DGP
set seed 42
clear
set obs 1000

* True model: y = 2*x + eps,  eps ~ N(0, 1)
generate x = rnormal()
generate eps = rnormal()
generate y = 2*x + eps

regress y x
display "Estimated beta = " _b[x] " (true is 2.0)"
```

Graphics

Three commands cover the everyday cases, rendered by `tea`'s own dependency-free SVG engine:

```
scatter y x   [if] [in] [, title() xtitle() ytitle() saving(FILE) noview]
line    y x   [if] [in] [, sort  ...same options...]
histogram v   [if] [in] [, bins(#) freq ...same options...]
```

Output is a vector SVG — publication quality by default, and identical byte-for-byte across platforms (plots are covered by golden-file regression tests). In the interactive REPL the plot opens in your OS viewer; `saving(FILE)` writes to a named file instead, and `noview` suppresses the viewer. In do-files plots are only ever written to files, never opened, so batch output stays deterministic.

`line` connects points in data order; add `sort` to order by x. `histogram` shows density by default (`freq` for counts); automatic binning is `min(ceil(sqrt(N)), 50)`. In the browser edition plots appear in the panel beside the terminal.

twoway — overlaid series

The Stata `twoway` grammar composes several series in one plot, each with its own `if` filter and styling:

```
twoway (TYPE y x [if], series-options) ... [, global-options]
```

where `TYPE` is `scatter`, `line`, `connected`, or `lowess`. Series options: `lcolor()` / `mcolor()`, `lpattern(solid|dash|dot|...)`, `msymbol(i)` (invisible marker — labels only), `mlabel(strvar)`, `mlabcolor()`, `mlabposition(#)` (clock positions), and for lowess `bwidth(#)` (default 0.8), `mean`, `adjust`. Global options: `title()`, `xtitle()`, `ytitle()`, `note()`, `legend(off)`, `yline(# [, lpattern() lcolor()])` (repeatable), `yscale(range(lo hi))`, `ylabel(a(step)b)` / `xlabel(a(step)b)`, `name(NAME[, replace])`, and `saving(FILE)`. As in Stata, axis titles given inside a series apply to the whole graph. A simple legend is drawn for multi-series plots unless `legend(off)`.

`lowess` is locally weighted regression: tricube kernel, running-line fit (option `mean` for a running mean), bandwidth as a fraction of the sample, `adjust` rescaling the smooth to the mean of y.

Tick rules and plot range are separate, as in Stata: `ylabel(0(.5)2.5)` fixes where the ticks sit, but the plot range always extends to cover the data — a label rule never clips observations.

```
twoway (line TFR year if ISO=="KOR", lcolor(blue)) ///
       (lowess TFR year if ISO=="KOR"), ///
       yline(2.1, lpattern(dot)) legend(off) name(korea, replace)
```

graph box — grouped box plots

```
graph box y [if] [in], over(v1[, subopts]) [over(v2[, subopts])]
    [noout] [title() note() name() saving()]
```

Boxes show the median and p25–p75 (Stata's percentile interpolation); whiskers reach the adjacent values (most extreme observations within 1.5 IQR of the box); points beyond are drawn as outside values unless `noout`. With two `over()` levels the first varies fastest inside bands formed by the second, Stata's layout. Group names honor attached value labels; `relabel(# "txt" ...)` renames positionally, and `label(angle(#) labsize(vsmall|small|...|vlarge))` styles the tick text. Unknown *cosmetic* suboptions inside graphics options are accepted and ignored (see COMPATIBILITY.md); structural errors — an unknown plot type, a missing variable, a malformed series — still fail loudly.

Named graphs and graph combine

`name(NAME[, replace])` stores the finished graph in an in-session registry *and* writes `NAME.svg` in the working directory (a deliberate deviation from Stata, where graphs live only in memory: on a CLI the file is the artifact, and in the browser each write feeds the Plots panel). Reusing a name without `replace` is error 110.

```
graph combine N1 N2 ... [, cols(#) rows(#) title() note() name() saving()]
```

composes stored graphs in a grid — panels nest as scaled SVG, so the result stays a crisp vector. `graph dir` lists the registry; `graph drop NAME|_all` clears it.

A worked example on bundled data:

```
sysuse grunfeld
scatter invest value, title("Investment vs firm value") ///
    xtitle("Market value") ytitle("Gross investment") saving(grun.svg)
```

SVG drops directly into LaTeX documents (via the `svg` package, or one `rsvg-convert` step to PDF).

Bundled practice datasets (sysuse)

Every bundled dataset carries variable labels (`describe` after any `sysuse` shows them; the WEO extract's 145 indicators are labeled from the IMF subject descriptions). The label tables live beside the CSVs as `data/NAME.lbl` and are embedded at generation time.

Six datasets are embedded inside the binary itself — no files to install, no network, identical in the browser edition:

```
. sysuse dir
  grunfeld  Grunfeld (1958) investment panel: 10 US firms x 1935-1954 (xtreg, hausman)
  airline   Box-Jenkins airline passengers, monthly 1949-1960 (tsset, arima)
  longley   Longley (1967) US macro, 16 obs: famously ill-conditioned (regress)
  nmes1988  Deb-Trivedi (1997) medical care demand, 4406 persons 66+ (poisson, logit)
  pwt       Penn World Table 10.0 sample: 22 countries x 1950-2019, CC BY 4.0 (growth)
  weo       IMF World Economic Outlook, April 2026: 197 countries + 13 aggregates, 145 indicators
```

Load one with `sysuse NAME[, clear]` — the same unsaved-data guard as `use` applies.

The WEO database

`weo` is the complete published IMF World Economic Outlook database, April 2026 vintage, as a long panel: `country iso year aggregate` plus one variable per WEO subject code, lowercased. The codes are the IMF's own unambiguous names — `ngdp_rpch` is real GDP growth, `pcpipch` CPI inflation (period average), `lur` the unemployment rate, `bca_ngdpd` the current account in percent of GDP, `ggxwdg_ngdp` general government gross debt in percent of GDP, and so on. The full code → descriptor → units table ships as `data/weo_codes.txt`.

World and regional groups (World, Advanced Economies, G7, Euro Area, EU, Emerging Market and Developing Economies, regional aggregates) carry `aggregate==1`; countries carry `aggregate==0`:

```
sysuse weo
keep if aggregate==0            // the 197-country panel
xtset iso year
xtreg ngdp_rpch ggxwdg_ngdp if year<=2025, fe

sysuse weo, clear
keep if aggregate==1            // the 13 groups
line ngdp_rpch year if iso=="G001", sort title("World real GDP growth")
```

101 of the 145 subject codes are published for groups only and are empty on country rows — faithful to how the IMF publishes the database. Years through 2031 are the projections *as published in this vintage*:

the bundled copy is a practice snapshot, not a live data service. Cite as *IMF, World Economic Outlook Database, April 2026.*

The other five

grunfeld is the canonical panel-methods teaching dataset; `xtreg invest value capital, fe` reproduces the textbook estimates (0.110, 0.310). **airline** is the Box–Jenkins monthly series every ARIMA text uses. **longley** is the famously ill-conditioned regression used to certify least-squares implementations — `regress employed gnpdeflator gnp unemployed armedforces population year` reproduces the certified solution. **nmes1988** is Deb and Trivedi's (1997, *JAE*) medical-care demand microdata: physician-visit counts, health status, insurance and income for 4,406 persons 66+ — real material for `poisson` and `logit`. **pwt** is a Penn World Table 10.0 sample (CC BY 4.0) for growth regressions.

Provenance, licenses, and citations for all six are in `data/SOURCES.md`. To refresh the embedded data (for example a newer WEO vintage): run `tools/curate_weo.py` on the new download, then `tools/gen_sysdata.py`, and rebuild — the curator auto-detects both the classic bulk-file and the data-portal export formats.

The browser edition

The WebAssembly build at https://micomrkaic.github.io/tea/ is the same program compiled for the browser: same commands, same estimators, same bundled data, same output to the last byte — the full regression suite runs inside the browser build and passes identically. Nothing you load or type leaves your machine; there is no server side.

Practical differences from the native binary:

- **Files** live in an in-browser filesystem. The *Upload data file* button places files where `use` / `import delimited` can see them; *Download workspace files* retrieves anything you `save` or `export`.
- **Plots** render in the panel beside the terminal instead of opening an OS viewer.
- **`shell`** is unavailable (browsers have no processes) and fails with a clear message.
- The terminal offers the same readline-style editing and Tab completion as the native REPL.

The first visit downloads about 2 MB (compressed); the browser caches it afterwards. The browser edition is a demonstration and practice channel — for production work, build the native binary.

Backend-independent output

`tea` promises that **the same do-file prints byte-identical output on every machine, operating system, CPU, and BLAS library**. For well-posed computations this is automatic — results agree to displayed precision everywhere. The danger zone is *mathematical zeros*: the residual sum of squares of a perfect fit, or the standard error of an exactly-determined coefficient, which different numerical backends render as different sub-epsilon noise (`5.9e-29` on one machine, `4.1e-29` on another), occasionally flipping what gets printed entirely (an SE of `0` versus `1e-16` turns `t = 0.00, p = 1.000` into `t = 1e+16, p = 0.000`).

`tea` therefore snaps such quantities to exactly zero when they fall below tight *relative* tolerances: residual SS under 1e-12 of total SS, and the consequences that follow — residual vectors, standard errors, coefficients under 1e-8 of the largest in an exact fit, `sigma_u` when panel intercepts are identical, and the `hausman` difference matrix at machine precision. A perfect fit reports `F = inf`, `p = 0.0000`, deterministically. **Normal estimation results are untouched to the last bit** — the thresholds fire only on genuine degeneracy, and a quantity that is merely small (rather than a mathematical zero) is left alone by the relative tests.

This is verified, not asserted: the regression suite passes byte-identically under native gcc + OpenBLAS (release and sanitizer builds) and under WebAssembly clang + reference BLAS + musl — two maximally different numerical stacks. The tradeoff, stated plainly: raw sub-epsilon values in degenerate fits are not displayed. The reasoning is documented in the README's "Semantics decisions" and in this manual so users of published results can account for it.

Escape hatches: when to leave `tea`

`tea` is excellent at the data-prep-to-quick-estimate loop, but many things are out of scope. The escape pattern is:

1. Do data prep in `tea`.
2. `export delimited` to CSV.
3. `shell python myscript.py` (or R, or anything).
4. `import delimited` the results back.

When to escape

What you need	Where to go
Plotting	`gnuplot`, Python's `matplotlib`, R's `ggplot2`
GARCH / EGARCH / TGARCH	Python `arch`, R `rugarch`
Exact-ML ARIMA (Kalman)	R's `arima()`, `forecast::Arima`
Seasonal ARIMA	R's `arima()` with `seasonal` arg
Bayesian inference	PyMC, Stan (`rstan`, `cmdstanpy`)
Mixed-effects models	R's `lme4`, Python `statsmodels.MixedLM`
Survival analysis	R's `survival`, Python `lifelines`
Cointegration / VAR / VECM	R's `urca`, `vars`
Machine learning	`scikit-learn`, `xgboost`, `lightgbm`
Network / graph models	`networkx`, R `igraph`
Spatial econometrics	R's `spdep`, Python's `pysal`

A bridge example

```
* Estimate panel FE in tea, then use Python for a residuals plot
xtset country year
xtreg gdp_growth L.gdp_growth inflation, fe
predict resid, e

* Export for plotting
keep country year resid
export delimited "resid.csv", replace

* Hand off to Python (assuming plot_residuals.py exists)
shell python plot_residuals.py resid.csv resid_plot.png
```

Known limitations

These are not bugs in the strict sense — they're scope decisions for v1.0 that we may revisit based on testing.

Data and metadata

- **.dta round-trip does not preserve xtset state.** After `save mydata.dta` then `use mydata.dta`, the panel/time variable assignment is lost. Workaround: re-run `xtset country year` after `use`. This is the standard Stata-do-file convention anyway.
- **Variable name length cap of 32 characters.** Factor-variable interaction names like `2010.year#1985.country#c.gdp` (28 chars) fit; longer 3-way interactions truncate.

Econometric estimators

- **ARIMA is exact ML** (state-space engine, since v1.6.37); standard errors are OIM where Stata defaults to OPG.
- **No seasonal ARIMA** (`sar`, `sma` terms).
- **ARIMA MA standard errors** have a 1–5% precision loss from the numerical-Hessian computation. Point estimates are unaffected.
- **`xtreg, be` uses straight OLS** on panel means. Stata's default applies a slight variance correction for unbalanced panels (down-weighting groups with few observations). Differences appear only on unbalanced panels and only in the SEs (point estimates are identical).
- **`xtabond` is difference GMM only** (v1.6.42): `lags(1)`, no system GMM (`xtdpdsys`), Windmeijer correction and AR tests staged.
- **VAR yes, VECM no.** `var/vargranger/irf/lpirf` and the Johansen rank test (`vecrank`) are implemented; full VECM estimation still means escaping to R.

Factor variables

- **No composition with TS operators.** Tokens like `i.country#c.L.gdp` are not parsed. Workaround: pre-compute the lag with `gen L1_gdp = L.gdp` and use `c.L1_gdp`.
- **`i.var` requires numeric var.** Use `encode` on string identifiers.
- **Cell-count cap of 10 000.** Larger factor expansions are blocked.

Programming

- **No `program define`, no `syntax` parser, no Mata.** You can write loops and use macros, but you cannot define new commands.

Graphics

`scatter`, `line`, `histogram`, multi-series `twoway` (with lowess), `graph box` with two-level `over()`, and `graph combine` cover the everyday cases (see the Graphics chapter). There is no faceting/`by()` graphics, no bar/pie charts, and no graph editor; for figures beyond these, `export delimited` and pipe to Python / R / gnuplot.

Reporting bugs

If you find a bug:

1. Distill it into a small reproducer — a do-file plus a small input file is ideal. If the input data is sensitive, replace it with `set seed N; gen x = rnormal()` or similar synthetic.
2. Note your `tea --version`, OS, and compiler version.

3. Send to the author at the email associated with your distribution of `tea`. If you don't have one, ask the person who gave you the software.

What counts as a bug:

- Wrong numerical answer (with hand-calculated expected value).
- Crash, segfault, or hang on input that should work.
- Different behavior from documented Stata semantics (when not in the "known limitations" list above).

What does not count as a bug:

- Missing feature (those go on the feature-request list).
- Disagreement with Stata's exact numerical output to many decimal places (small differences arise from BLAS vendor variation and are not correctness issues).
- Things listed in the "Known limitations" chapter.

Function reference

Functions available in any expression context (`generate`, `replace`, `if` clauses, `display`, ...). Each entry shows the signature and a one-line description. In all cases: a missing argument propagates to a missing result, unless otherwise stated.

Arithmetic and rounding

Signature	Returns
`abs(x)`	$\lvert x \rvert$
`int(x)`	truncate toward zero
`floor(x)`	largest integer $\leq x$
`ceil(x)`	smallest integer $\geq x$
`round(x)`	round to nearest integer (half away from zero)
`round(x, unit)`	round to nearest multiple of `unit`
`sign(x)`	-1 / 0 / $+1$
`mod(x, y)`	$x \bmod y$, same sign as x
`min(x, y, ...)`	minimum of arguments (missing skipped)
`max(x, y, ...)`	maximum of arguments (missing skipped)
`cond(test, a, b)`	`a` if `test` is true, else `b`

Exponential and logarithm

Signature	Returns
`exp(x)`	e^x
`ln(x)`	natural log; missing if $x \leq 0$
`log(x)`	alias for `ln`
`log10(x)`	base-10 log; missing if $x \leq 0$
`sqrt(x)`	$\sqrt{x}$; missing if $x < 0$

Trigonometric and hyperbolic

Signature	Returns
`sin(x)`, `cos(x)`, `tan(x)`	radians
`asin(x)`, `acos(x)`	inverse trig; range checked
`atan(x)`	inverse tangent
`atan2(y, x)`	two-argument inverse tangent
`sinh(x)`, `cosh(x)`, `tanh(x)`	hyperbolic

Probability distributions

Signature	Returns
`normal(z)`	standard-normal CDF $\Phi(z)$
`normalden(z)`	standard-normal PDF $\phi(z)$
`normalden(z, mu, sd)`	normal PDF with mean and SD
`lnnormal(z)`	$\log \Phi(z)$, stable in the lower tail
`lnnormalden(z)`	$\log \phi(z)$, stable in extremes
`invnormal(p)`	inverse CDF $\Phi^{-1}(p)$
`ttail(df, t)`	upper-tail Student-t: $P(T > t)$
`invttail(df, p)`	inverse: t such that $P(T > t) = p$
`F(df1, df2, F)`	lower-tail F CDF
`Ftail(df1, df2, F)`	upper-tail F: $P(F > f)$
`chi2(df, x)`	lower-tail χ^2 CDF
`chi2tail(df, x)`	upper-tail χ^2: $P(X > x)$

Random number generation

Reproducible from `set seed N`. See the “Random numbers and Monte Carlo” chapter.

Signature	Returns
`runiform()`	uniform $[0, 1)$
`rnormal()`	standard normal
`rnormal(mu)`	normal with mean `mu`, sd 1
`rnormal(mu, sd)`	normal with mean `mu` and SD `sd`

Missing-value tests and ranges

Signature	Returns
`missing(x)`	1 if `x` is any missing code, else 0
`inrange(x, lo, hi)`	1 if `lo` $\leq x \leq$ `hi`
`inlist(x, v1, v2, ...)`	1 if `x` equals any of `v1`, `v2`, ...

String construction and inspection

Signature	Returns
`strlen(s)`, `length(s)`	character count of `s`
`substr(s, start, len)`	substring; `start` is 1-based; `-1` for last
`strpos(s, sub)`	1-based index of `sub` in `s`, 0 if absent
`strrpos(s, sub)`	like `strpos` but search from the right
`word(s, n)`	the nth whitespace-separated word
`wordcount(s)`	number of whitespace-separated words
`abbrev(s, n)`	abbreviate `s` to width `n`
`char(n)`	single-character string for ASCII code `n`

String transformation

Signature	Returns
`strupper(s)`, `upper(s)`	uppercase
`strlower(s)`, `lower(s)`	lowercase
`strproper(s)`, `proper(s)`	title case (first letter of each word)
`strtrim(s)`, `trim(s)`	strip leading and trailing whitespace
`strtoname(s)`	make a valid variable name: invalid chars to `_`, leading digit gets `_` prefix (`"CC.EST"` -> `CC_EST`)
`ltrim(s)`, `rtrim(s)`	strip leading / trailing whitespace
`itrim(s)`	collapse internal whitespace runs to one space
`strreverse(s)`	reverse character order
`subinstr(s, from, to)`	replace all occurrences of `from` with `to`
`subinstr(s, from, to, n)`	replace at most `n` occurrences
`subinword(s, from, to)`	replace whole-word matches only
`strmatch(s, pattern)`	glob-style match (`*`, `?`); 1 / 0

Regular expressions (POSIX extended)

Signature	Returns
`regexm(s, pat)`	1 if `pat` matches anywhere in `s`, else 0; populates `regexs()`
`regexs(n)`	nth captured group from last `regexm` (0 = full match)
`regexr(s, pat, repl)`	replace first match in `s`; `repl` may use `\1` ... `\9`

Numeric ↔ string conversion

Signature	Returns
`string(x)`	default decimal string
`string(x, fmt)`	format `x` with the given Stata format
`strofreal(x)`, `strofreal(x, fmt)`	alias for `string`
`real(s)`	parse `s` as numeric; missing on failure

Calendar: building dates

Signature	Returns
`mdy(month, day, year)`	daily date (%td) for (month, day, year)
`ym(year, month)`	monthly date (%tm)
`yq(year, quarter)`	quarterly date (%tq)
`yh(year, half)`	half-yearly date (%th)
`yw(year, week)`	weekly date (%tw)
`date(s, fmt)`	parse string `s` per `fmt` (`"YMD"`, `"DMY"`, `"MDY"`, etc.); `daily()` is the Stata alias
`quarterly(s, "YQ")`	parse "2020-Q3" to a quarterly date; likewise `monthly(s,"YM")`, `halfyearly(s,"YH")`, `weekly(s,"YW")`, `yearly(s,"Y")`. Out-of-range periods give missing

Calendar: extracting components from a daily date

Signature	Returns
`year(d)`	calendar year
`month(d)`	month, 1–12
`day(d)`	day of month, 1–31
`quarter(d)`	quarter, 1–4
`halfyear(d)`	half-year, 1 or 2
`week(d)`	ISO week number
`dow(d)`	day of week, 0 (Sun) – 6 (Sat)
`doy(d)`	day of year, 1–366

Calendar: converting between frequencies

Signature	Returns
`mofd(d)`	monthly date corresponding to daily date `d`
`dofm(m)`	first day of month `m` (as daily)
`qofd(d)`	quarterly index of daily `d`
`dofq(q)`	first day of quarter `q`
`hofd(d)`	half-year index of daily `d`
`dofh(h)`	first day of half-year `h`
`wofd(d)`	weekly index of daily `d`
`ty(y)`	year-as-yearly-date (just y, for display)
`dofy(y)`	first day of year `y`
`tm`, `tq`, `tw`, `th`, `td`	identity converters used for format-tagging

Special: running and time-series-context

Signature	Returns
`sum(x)`	running cumulative sum (resets per by-group)
`_n`	current observation index (1-based)
`_N`	total observation count in current scope
`x[_n-1]`, `x[_n+1]`	lag / lead within current sort order
`x[1]`, `x[_N]`	first / last observation of current scope

egen function reference

Functions usable as `egen newvar = func(...)`; see the `egen` part of the Data-management chapter.

Group-wise aggregators

When combined with `by(grouplist)`, these compute one value per group and broadcast that value back to every row in the group. Without `by()`, they collapse to a single scalar broadcast everywhere.

Signature	Returns
`mean(exp)`	arithmetic mean
`sum(exp)`, `total(exp)`	sum (missing values skipped)
`count(exp)`	number of non-missing observations
`min(exp)`, `max(exp)`	extrema
`sd(exp)`	sample standard deviation, $n-1$ denominator
`var(exp)`	sample variance
`median(exp)`	50th percentile (linear interpolation)
`iqr(exp)`	interquartile range, 75th − 25th
`pc(exp)`	percentile; option `p(N)` sets which (default 50)

Position and identity

Signature	Returns
`rank(exp)`	1-based rank within group; ties handled by mean rank
`group(varlist)`	1. . . K integer ID per unique combination of `varlist`
`tag(varlist)`	1 on the first row of each `varlist` group, 0 otherwise

Row-wise aggregators (across columns of a single row)

These ignore `by()` (the row is the unit).

Signature	Returns
`rowtotal(varlist)`	sum across columns; missing skipped
`rowmean(varlist)`	mean across columns
`rowmin(varlist)`	minimum across columns
`rowmax(varlist)`	maximum across columns
`rowsd(varlist)`	sample SD across columns
`rowmiss(varlist)`	count of missing cells in the row
`rownonmiss(varlist)`	count of non-missing cells

Summarize / tabstat statistics keywords

Available with `tabstat varlist, statistics(...)`, and many also as percentile selectors after `summarize, detail`.

Keyword	Meaning
`n`	non-missing count

Keyword	Meaning
`mean`	arithmetic mean
`sum`	total
`sd`	standard deviation (sample, $n-1$)
`variance`	variance
`min`, `max`	extrema
`range`	`max - min`
`median`	50th percentile
`p1`, `p5`, `p10`, `p25`, `p50`	percentiles
`p75`, `p90`, `p95`, `p99`	percentiles
`iqr`	75th − 25th percentile
`skewness`, `kurtosis`	after `summarize, detail`

Stata cheat sheet

A quick reference for Stata users adapting to `tea`.

Same as Stata

- Syntax: `[prefix:] command [varlist] [if] [in] [weight] [, options]`
- Missing-value algebra, including `if x > 5` including missing
- Macros: `` `local' `` and `$global`, `r()` and `e()` returns
- `by:`, `bysort:`, the `[_n-1]` subscript, `_N`
- Value labels: `label define / values / list`
- Factor variables (`i.`, `c.`, `ib<n>.`, `#`, `##`)
- Time-series operators (`L. F. D. S.`)

Tea-specific differences

- `m:m` merge is rejected with an explanatory message.
- `xtreg, be` uses simple OLS on panel means.
- `arima` is exact ML (v1.6.37+); SEs are OIM (Stata: OPG).
- `i.country#c.L.gdp` doesn't parse; pre-compute the lag.
- No `.gph` graphics — use `export` and external tools.
- `program define`, `syntax`, and Mata are absent.

Stata commands that don't exist in tea

- GMM beyond `xtabond`: `gmm`, `xtdpd`, `xtdpdsys`.
- Time series: `vec` (rank via `vecrank` only), `dfgls`, `xcorr`, `wntestq`, `arch`.
- Survival: `stcox`, `streg`, `stset`.
- Mixed: `mixed`, `xtmixed`, `meqrlogit`.
- Bayesian: `bayes:`, `bayesmh`.

- Multivariate: mvreg, factor, pca.
- Multiple imputation: mi.

For each of these, see the "Escape hatches" chapter for the recommended external tool.

Command reference

Generated from the tea binary by `tools/gen_cmdref.sh` *— this text is exactly what* `help CMD` *prints at the prompt, and regenerating it after any change keeps the manual and the implementation in agreement.*

twoway

```
twoway (TYPE y x [if], opts) ... [, gopts]  overlay plot; TYPE: scatter line connected lowess
    per-series: lcolor() lpattern() msymbol(i) mlabel(var) mlabposition(#) bwidth() mean adjust
    global: title() xtitle() ytitle() note() legend(off) yline(#,..) yscale(range()) ylabel(a(s)b) n
```

graph

```
graph box y [if], over(v[,sub]) [over(v2)] [noout] ...   grouped box plots
    graph combine N1 N2 ... [, cols(#) title() note() name()]   compose named graphs
    graph dir | graph drop NAME|_all                            registry
```

scatter

```
scatter yvar xvar [if] [in] [, title() xtitle() ytitle() saving() noview]
    SVG scatter plot; saving(FILE) writes to FILE instead of tea_graph.svg
    e.g.  scatter gdp_growth inflation if year>2000, title("Growth vs inflation")
```

line

```
line yvar xvar [if] [in] [, sort title() xtitle() ytitle() saving() noview]
    SVG line plot; connects points in data order (use sort to order by x)
    e.g.  line gdp year if country=="US", sort
```

histogram

```
histogram var [if] [in] [, bins(#) freq title() saving() noview]
    SVG histogram; density by default, freq for counts, auto bins = min(ceil(sqrt(N)),50)
    e.g.  histogram wage, bins(30) freq
```

generate

```
generate [type] newvar = exp [if] [in]      create a new variable
    e.g.  gen logy = log(income) if year >= 2000
```

replace

```
replace var = exp [if] [in]                  modify existing variable
    e.g.  replace income = . if income < 0
```

egen

```
egen newvar = fn(args) [, by(varlist)]       extended generate (mean/sum/group/...)
    e.g.  egen meanY = mean(y), by(country)
```

list

```
list [varlist] [if] [in]                     print observations
    e.g.  list country year gdp if year>2020 in 1/10
```

summarize

```
summarize [varlist] [if] [in] [weight] [, detail]      N/mean/sd/min/max; sets r()
    e.g.  summarize gdp_growth, detail
```

tabstat

```
tabstat varlist [if] [in], statistics(stat ...) [by(var)] [columns(stat|var)] [format(%9.2f)]
    by() groups show value labels when attached; format() styles every cell incl. Total
    stats: mean sd var cv min max range sum count median p1-p99 iqr
    e.g.  tabstat gdp pop, stats(mean sd p25 p50 p75) by(region)
```

count

```
count [if] [in] [fw=]                        count matching observations
```

describe

```
describe [varlist]                           show variables, types, formats
```

drop

```
drop varlist | if exp | in range             remove vars or observations
    e.g.  drop if gdp == .    |   drop tempvar1 tempvar2
```

keep

```
keep varlist | if exp | in range             complement of drop
    e.g.  keep country year gdp
```

rename

```
rename oldname newname                       rename a variable
    e.g.  rename _C* C*   (wildcard rename: strip leading underscore)
```

order

```
order varlist                                reorder variables (listed ones go first)
    e.g.  order country year
```

aorder

```
aorder [varlist]                             alphabetize variable order
```

label

```
label variable v "text"                      attach a description to a variable
  label define lblname # "text" # "text"   define a value-label set
  label values varlist lblname             attach value labels to variables
```

format

```
format varlist %fmt                          set display format
    e.g.  format date %td   |   format gdp %12.2f
```

sort

```
sort varlist                                 ascending stable sort
    e.g.  sort country year
```

gsort

```
gsort [+-]var [+-]var ...                    sort with per-key direction
    e.g.  gsort -gdp +country  (gdp desc, then country asc)
```

tabulate

```
tabulate var [if] [in] [weight]              one-way frequency table
    e.g.  tabulate region if year == 2020
```

tsset

```
tsset timevar [, format(%fmt)]               declare time series
    e.g.  tsset year
```

xtset

```
xtset panelvar timevar                       declare panel (gap-aware L./F./D./S.)
    e.g.  xtset country year
```

xtdescribe

```
xtdescribe                                   summarize panel structure
    e.g.  xtdescribe   (requires xtset first; shows n, T, balance)
```

xtreg

```
xtreg y x1 x2 ... [if] [in], fe [vce(robust|cluster v)]   panel FE OLS
    e.g.  xtreg growth L.growth pop, fe
          (requires xtset; within estimator; reports R-w/b/o, sigma_u/e, rho)
```

hausman

```
hausman                                       FE vs RE specification test
    e.g.  xtreg y x, fe
          xtreg y x, re
          hausman   (Ho: cov(u_i, x) = 0; rejection means use FE)
```

logit

```
logit y x1 x2 ... [if] [in] [weight], [vce(robust|cluster v)]   logistic regression
    e.g.  logit voted income education, vce(cluster county)
```

probit

```
probit y x1 x2 ... [if] [in] [weight], [vce(robust|cluster v)]   probit regression
    e.g.  probit voted income education, vce(robust)
```

ivregress

```
ivregress 2sls y [exog] (endog = instruments) [if] [in] [weight] [, vce(robust|cluster v)]
    Two-stage least squares with first-stage F diagnostic.
    e.g.  ivregress 2sls wage educ (exper = age momeduc daddyEduc)
```

poisson

```
poisson y x1 x2 ... [if] [in] [weight] [, vce(robust|cluster v)]   poisson regression
    e.g.  poisson accidents speed_limit population, vce(cluster state)
```

arima

```
arima y [exog] [if] [in], arima(p d q) [sarima(P D Q s) noconstant]
    exact-ML ARIMA/ARMAX; sarima() = multiplicative seasonal terms
    e.g.  arima gdp, arima(1 1 1)   arima lp, arima(0 1 1) sarima(0 1 1 12)
          arima cpi unemp, arima(2 0 1)        ARMAX(2,1) with exog regressor
```

margins

```
margins , dydx(*|varlist) [atmeans]            average marginal effects
    e.g.  logit y x1 x2
          margins, dydx(*)              AME for all regressors
          margins, dydx(x1) atmeans     MEM for x1 at sample means
```

estimates

```
estimates store|restore|dir|drop|table          named estimates ledger
    e.g.  regress y x1 x2
          estimates store m1
          regress y x1 x2 x3
          estimates store m2
          estimates table m1 m2, se star  stats(N r2 rmse)
```

est

```
est = estimates  (short form)                   see `help estimates`
```

estout

```
estout [names] [, format(latex|markdown|plain) se|t|p stars stats(...)]
    Side-by-side LaTeX/markdown/plain table of stored estimates.
    e.g.  estout m1 m2, stats(N r2 rmse) using table.tex
```

collapse

```
collapse (stat) v1 v2 ... , by(g) [weight]   aggregate to groups
    e.g.  collapse (mean) gdp pop, by(region year)
```

merge

```
  merge 1:1|m:1|1:m keyvars using FILE [, ...] join master with using file
      e.g.  merge 1:1 country year using gdp.tea   (m:m is rejected by design)
```

reshape

```
Sort, reshape & combine
  sort varlist                                 ascending stable sort
  gsort [+-]var [+-]var ...                    sort with per-key direction
  reshape long|wide stubs , i(idvars) j(jvar)  pivot wide<->long (in-place)
  merge 1:1|m:1|1:m keyvars using FILE [, ...] join master with using file
  collapse (stat) v1 v2 ... , by(g) [weight]   aggregate to groups
'help CMD' shows a command's full syntax line(s).
```

recode

```
  recode varlist (rules) [, gen(stub)]         map values
      e.g.  recode score (1/3=1)(4/6=2)(7/10=3) (missing=.), gen(score_g)
```

encode

```
  encode strvar, generate(newvar) [label(name)]  map strings to integers + value label
```

decode

```
  decode intvar, generate(newvar)              inverse of encode (codes back to strings)
```

destring

```
  destring varlist, {replace|generate(stub)} [force] [ignore("chars")]   str -> num
      e.g.  destring z4, replace force   (junk values become missing)
```

tostring

```
  tostring varlist, {replace|generate(stub)} [force] [format(%fmt)]      num -> str
```

codebook

```
  codebook [varlist]                           type, uniques, missing, range
```

import

```
  import excel FILE [, sheet(name) cellrange(A17[:AF22000]) firstrow case(preserve|lower|upper) clear]
      cellrange restricts to a sheet rectangle; with firstrow its first row is the header row
import delimited FILE [, delimiters(...) rowrange(r1[:r2]) colrange(c1[:c2]) case(lower|preserve|upper
      e.g.  import excel "WPP2024.xlsx", sheet("Estimates") cellrange(A17:AF22000) firstrow case(lower
```

export

```
  export delimited using FILE                  write CSV/TSV
      e.g.  export delimited using out.csv
```

save

```
save FILE [, replace]                        write Stata .dta (default) or .tea
    e.g.  save mydata.dta, replace        - emits Stata-compatible .dta
    e.g.  save mydata.tea, replace        - native tea binary
```

outreg2

```
outreg2 using FILE [, replace|append ctitle() dec() bdec() se label
    symbol() alpha() addstat("Name", expr, ...) addtext() addnote()]
    regression-table exporter (tab-separated; opens in Excel)
```

which

```
which CMD  - report whether CMD is a built-in tea command
```

ssc

```
ssc install PKG  - accepted and skipped (no package system)
```

isid

```
isid varlist  - error 459 unless varlist uniquely identifies obs
```

duplicates

```
duplicates report|drop [varlist]
```

tempfile

```
tempfile NAME...  - set local macros to fresh temp-file paths
```

tempname

```
tempname NAME...  - set local macros to fresh scratch names
```

constraint

```
constraint [define] # expr = expr | list | drop #|_all      linear constraints
```

dfactor

```
dfactor (y1 y2 ... [= exog][, noconstant ar(1)]) (f1 [f2..] = , ar(#))
    dynamic factors; obs-eq ar(1) = idiosyncratic AR errors; smfactor(stub)
```

areg

```
areg y x.., absorb(v) [robust cluster(v)]          absorbed fixed effects
```

xtabond

```
xtabond y [x..], lags(1) [twostep robust maxldep(#)] Arellano-Bond GMM
```

xtivreg

```
xtivreg y [x..] (endo = inst..), fe [robust]       panel IV (within)
```

sspace

```
sspace (s L.s.., state [noerror]).. (y s.. [, noerror nocons])..
    , constraints(#) covstate(identity|diagonal) [diffuse smstates(stub)]
    linear state-space models; diffuse = nonstationary initialization
```

ucm

```
ucm Y, model(llevel|lltrend|rwalk|rwdrift|ntrend) [seasonal(#) cycle smstate(NEW)]
    unobserved components; cycle = damped stochastic cycle (damping+frequency)
```

var

```
var Y1 Y2 ...[, lags(1/p)]                  reduced-form VAR
```

vargranger

```
vargranger                                   Granger Wald tests after var
```

irf

```
irf create[, step(#)] | irf table [irf|oirf] impulse responses after var
```

lpirf

```
lpirf VAR[, step(#) lags(#)]                local-projection IRFs (Newey SEs)
```

vecrank

```
vecrank Y1 Y2 ...[, lags(#)]                Johansen trace test
```

newey

```
newey Y XS [if] [in], lag(#)                 OLS with Newey-West (HAC) SEs
```

dfuller

```
dfuller VAR[, lags(#) trend drift noconstant regress]  augmented Dickey-Fuller
```

pperron

```
pperron VAR[, lags(#) trend noconstant regress]        Phillips-Perron unit-root test
```

tsfilter

```
tsfilter hp|bk|hamilton NEW = VAR[, opts]    business-cycle filters
```

ttest

```
ttest VAR == # | VAR1 == VAR2 | VAR, by(G)   t tests (one-sample, paired, two-sample)
```

correlate

```
correlate [varlist] [if] [in][, means covariance]  correlation (listwise)
```

corr

```
corr ...                                      abbreviation of correlate
```

pwcorr

```
pwcorr varlist  - pairwise correlation matrix
```

file

```
file open H using F, write [replace|append] | file write H "..." _n | file close H
```

confirm

```
confirm [new] file FILENAME | confirm [new|numeric|string] variable NAME
    error (601/602/111/110/7) unless the condition holds; use with
    capture:  capture confirm file f.dta
              if _rc { <create it> }
```

history

```
history [N] | history save FILE [, replace] | history clear
    list, export, or clear this session's interactive commands
    (native REPL arrow-key history persists in ~/.tea_history;
     the browser edition persists history in the browser)
```

sysuse

```
sysuse dir | sysuse NAME [, clear]
    load a practice dataset bundled inside the tea binary
    e.g.  sysuse grunfeld, clear
          xtset firm year
          xtreg invest value capital, fe
```

use

```
use FILE [, clear]                            read Stata .dta, .tea, .csv, or .tsv
    e.g.  use mydata.dta, clear             - extension dispatch decides format
    'use foo' with no extension looks for foo.dta (Stata default).
```

clear

```
clear                                         drop all data in current frame
```

error

```
error #                                       abort with return code # (Stata do-file assertions)
```

compress

```
compress                                      accepted for compatibility; tea storage is already mini
```

status

```
status                                        one-line summary: source, obs, vars, memory, sort/xtset
```

frame

```
frame create|change|copy|rename|put|drop|dir  multiple datasets
    e.g.  frame create alt   |   frame change alt   |   frame put x y, into(alt)
```

regress

```
regress y x1 x2 ... [if] [in] [weight] [, noconstant robust cluster(var)]   OLS
    e.g.  regress gdp_growth investment trade if year>=2000, cluster(country)
```

predict

```
predict newvar [, xb residuals]                                     predict from last fit
```

test

```
test v1 v2 ...                                                      Wald F-test (joint zero)
```

lincom

```
lincom <linear combo of coefs>                                     point estimate + SE of L'b
```

help

```
help [cmd]                                    list commands or show one's syntax
```

pwd

```
pwd                                           print working directory
```

cd

```
cd DIR                                        change working directory
```

mkdir

```
mkdir DIR [, recursive]                       create directory (recursive = mkdir -p)
```

dir

```
dir [pattern]                                 list files in current directory
```

rmdir

```
rmdir DIR                                     remove empty directory
```

erase

```
erase FILE  |  rm FILE                        delete a file
```

copy

`copy SRC DST [, replace]`	`copy a file`

do

`do FILENAME`	`run another do-file`

preserve

`preserve`	`snapshot the data in memory`

restore

`restore [, not preserve]`	`bring the preserve snapshot back`

version

`version`	`tea version & build info`

log

`log using FILE [, replace append] | log close`	`tee output to a file`

exit

`exit [, clear]`	`leave tea (clear drops data first)`

Native statements

Handled by the interpreter before command dispatch: `display`, `assert`, `shell` (and the `!cmd` escape), `#delimit`, `local`, `global`, `foreach`, `forvalues`, `while`, `if/else`, `capture`, `quietly`, `program define`, and comments (`*`, `//`, `///`).